

1. Guild of Heroes Manager

Players manage a guild of heroes in a fantasy city. Each hero (class `Hero`) has stats (HP, attack, level, salary) and is stored in an `ArrayList`. Players get quests (class `Quest`) of various difficulty and must assign suitable heroes. Success/failure is calculated based on stats, levels, and random chance. Players must balance gold income, hero fatigue, and guild reputation over a number of in-game weeks, with a simple main menu for “Hire Hero,” “Assign Quest,” “Rest,” and “View Status.”

Required implementation & minimum features:

- Use **at least 3 classes**: e.g. `Hero`, `Quest`, `Guild` (or `Game`), plus a `Main` class with `main()`.
 - Store heroes in an `ArrayList<Hero>` and quests in an `ArrayList<Quest>`.
 - Each hero must have **attributes**, **constructor**, and **methods** (e.g. `levelUp()`, `takeDamage()`, `getPower()`).
 - Implement a **main menu** in console (loop) with options like:
 - Hire hero
 - View heroes
 - View available quests
 - Assign hero to quest
 - Next day / next week
 - Implement quest resolution:
 - Compare hero power vs quest difficulty using simple formulas and randomness.
 - Update gold, hero HP/fatigue, and guild reputation.
 - Define a **clear win/lose condition**, e.g.:
 - Win: reach target reputation or gold after X weeks.
 - Lose: reputation or gold drops below zero.
-

2. Starport Tycoon

Players run a starport with ships arriving and departing. Each ship (`Ship` class) has type, cargo, and docking time. An `ArrayList` holds currently docked ships, queued ships, and historical logs. Each turn, players set docking fees, accept/reject ships, and manage limited docking bays. Events like “smuggler ship,” “VIP ship,” or “pirate threat” force decisions affecting reputation and income. The goal is to reach a profit or reputation target after a given number of days.

Required implementation & minimum features:

- Use **at least 3 classes**: `Ship`, `Starport`, and `Game` (or similar) plus `Main`.
 - Use `ArrayList<Ship>` for docked ships and waiting queue.
 - Each ship must have attributes like name, type, cargoValue, dockingTime, isSmuggler, etc.
 - Implement a **day-based or turn-based loop**:
 - Generate new ships each day.
 - Allow player to accept/reject ships depending on available docking slots.
 - Reduce docking time for ships already docked and depart them when done, adding income.
 - Implement a **simple reputation and income system**:
 - Certain ship types affect reputation (e.g., rejecting VIP or accepting smugglers).
 - Provide a **text-based menu** each day:
 - View docked ships
 - View queue
 - Set docking fee
 - End day
 - Include **end condition**:
 - Reach certain profit or survive a fixed number of days with positive reputation.
-

3. Survivor Colony: ASCII Outpost

Players run a small colony in a dangerous world. There are colonists (`Colonist` class), buildings (`Building` class), and resources stored in `ArrayLists`. Each turn, players assign colonists to tasks (gather food, build, defend). Random events (raids, storms, disease) change stats. There is a simple “turn system,” with the goal of surviving a fixed number of days or reaching a development level (e.g., build 5 buildings, reach 30 colonists).

Required implementation & minimum features:

- Use **at least 3 classes**: `Colonist`, `Building`, `Colony` (or `Game`), plus `Main`.
 - Store colonists and buildings in `ArrayList<Colonist>` and `ArrayList<Building>`.
 - Each turn (day):
 - Player chooses how many colonists to assign to each task (gather, build, defend, etc.).
 - Update resources (food, materials) based on assignments.
 - Implement **random events** (simple switch or if-else with `Random`):
 - E.g., raids, storms, sickness.
 - Update colonist count, resources, or building damage.
 - Show **status screen** each turn:
 - Number of colonists, buildings, food, materials, danger level.
 - Implement **lose condition** (e.g., all colonists dead or food ≤ 0) and **win condition** (e.g., survive X days or reach target size).
-

4. Detective Agency Chronicles

Players run a detective agency that receives cases. Each case (`Case` class) has difficulty, reward, suspects list, and clues stored in `ArrayList`. The player chooses which detective (`Detective` class) from their agency to assign based on skills (logic, stealth, persuasion). During a case, the game presents multiple-choice investigation steps in sequence. Choices affect the probability of solving the case. The assignment complexity: managing detectives, case queue, agency reputation, and funds over many cases.

Required implementation & minimum features:

- Use **at least 3 classes**: `Detective`, `Case`, `Agency` (or `Game`), plus `Main`.
 - Use `ArrayList<Detective>` for detectives and `ArrayList<Case>` for pending / solved cases.
 - Each case has:
 - Difficulty level
 - Reward money
 - A small list of suspects/clues (can be text only).
 - Implement a **menu system**:
 - View detectives
 - View available cases
 - Assign detective to case
 - Next day / process case
 - When processing a case:
 - Show a small sequence of choices (e.g., “Interrogate suspect” vs “Search location”).
 - Update success chance based on detective skills + chosen actions + randomness.
 - Maintain **agency money and reputation**:
 - Solved cases increase them, failed cases reduce them.
 - Define **end condition**: e.g., solve at least N cases or reach certain reputation; lose if money or reputation drops too low.
-

5. Arena of Champions (Turn-Based League)

Players create and manage a team of fighters (`Champion` class) that battle in a league. Champions have stats, skills, and levels, all stored in an `ArrayList`. The league schedule is generated, and each match is a turn-based battle where players choose moves for each champion. After each match, champions gain XP and can level up, increasing stats or unlocking new skills. The project can include a season system, standings table, and even a simple AI opponent team.

Required implementation & minimum features:

- Use **at least 3 classes**: `Champion`, `Team`, `League` (or `Game`), plus `Main`.
 - Use `ArrayList<Champion>` in each `Team`.
 - Implement basic **champion attributes**: HP, attack, defense, level, experience.
 - Create a few **predefined teams** or generate AI teams.
 - Implement a **turn-based battle system**:
 - Each turn: player chooses an action (attack, defend, special move).
 - Calculate damage and update HP.
 - Implement **XP and level up**:
 - After each battle, winning champions gain XP; reaching a threshold increases level and stats.
 - Simulate a **league/season**:
 - At least several matches with a simple standings table (wins/losses).
 - Define **winning condition** for the league (e.g., highest win count after all matches).
-

6. Dungeon Crawler: Party Edition

The player controls a party of adventurers (`PlayerCharacter` class) stored in an `ArrayList`, each with roles (warrior, mage, healer). The dungeon is represented as floors/rooms with enemies (`Monster` class) and loot items (`Item` class). Exploration is turn-based: move, fight, use item, rest. Combat uses simple formulas and random numbers. The aim: reach the last floor and defeat the boss.

Required implementation & minimum features:

- Use **at least 4 classes**: `PlayerCharacter`, `Monster`, `Item`, `Dungeon` (or `Game`), plus `Main`.
 - Use an `ArrayList<PlayerCharacter>` for the party and `ArrayList<Item>` for inventory.
 - Represent dungeon progression linearly (Floor 1 → Floor 2 → ...) or with simple room choices.
 - Implement **combat**:
 - At least two types of actions: normal attack and special/heal.
 - Calculate damage and handle KO/death.
 - Implement **loot & items**:
 - Monsters can drop items (healing potion, stat boost).
 - Player can use items in or out of combat.
 - Show **status screen** of party (HP, role, level) and dungeon progress.
 - Define **win condition** (defeat final boss) and **lose condition** (entire party dead).
-

7. Trading Card Battle Simulator

Players collect and battle with cards in a simplified TCG style. Each card (`Card` class) has attributes like name, attack, defense, and special effect text. A deck is an `ArrayList<Card>`. The game supports building a deck, then dueling an AI deck using simple rules (draw card, choose attack, resolve damage). Cards can be earned by winning matches.

Required implementation & minimum features:

- Use **at least 3 classes**: `Card`, `Deck`, `Game` (or `Battle`), plus `Main`.
 - Use `ArrayList<Card>` to represent a deck and hand.
 - Implement **deck management**:
 - Predefined set of available cards.
 - Player deck built from those cards (can be simple at start).
 - Implement a **battle system**:
 - Each turn: both players draw a card (if available).
 - Allow player to choose a card to attack.
 - Compare attack/defense and update some kind of life points or score.
 - Include a basic **AI opponent**:
 - Selects a card based on simple logic or random choice.
 - Implement **match result**:
 - Win/lose/draw based on life points or number of cards remaining.
 - Allow playing **multiple matches** and tracking total wins.
-

8. City Traffic Control Simulator

Players act as traffic controllers managing intersections in a small text-based city. Intersections (`Intersection` class) and roads (`Road` class) are tracked, and cars are simulated in an `ArrayList` of `Car` objects. Each “tick” of time, cars move along roads, and the player can change signals at intersections. The goal is to minimize average waiting time or avoid traffic jams.

Required implementation & minimum features:

- Use **at least 3 classes**: `Car`, `Intersection`, `Road`, plus `Main` and optionally `Game`.
 - Use `ArrayList<Car>` to represent cars currently in the system.
 - Implement a **time-step loop** (“tick”):
 - Each tick moves cars along roads.
 - Cars must stop at red lights and go at green lights.
 - Implement **menu/commands**:
 - Show current intersections and light status.
 - Change light cycles (e.g., switch North-South / East-West).
 - Track simple **statistics**:
 - Total cars passed, total waiting time, or max queue length.
 - Implement **end condition**:
 - Simulation runs for a fixed number of ticks, then shows a summary of performance.
-

9. Restaurant Empire Manager

Players run a chain of restaurants. Each restaurant (`Restaurant` class) has staff, menu items (`MenuItem` class), and daily income. Staff and menu are stored in `ArrayLists`. Players can hire/fire staff, add/remove menu items, set prices, and view daily reports (customer satisfaction, revenue, cost). Random events like “food critic visit” or “ingredient shortage” add variety. The goal might be to reach a target profit over 30 in-game days.

Required implementation & minimum features:

- Use **at least 3 classes**: `Restaurant`, `MenuItem`, `Staff` (or similar), plus `Main`.
 - Use `ArrayList<MenuItem>` and `ArrayList<Staff>` for each restaurant.
 - Implement **daily cycle**:
 - Calculate income and costs based on menu prices and staff wages.
 - Random events modify income or satisfaction.
 - Implement **menu management**:
 - Add/edit/remove menu items.
 - Change prices.
 - Implement **staff management**:
 - Hire/fire staff and pay their salary.
 - Show a **daily report**:
 - Revenue, costs, profit, and a simple satisfaction score.
 - Define **goal**: reach target total profit or survive N days without going bankrupt.
-

10. Space Exploration Fleet

Players control a fleet of ships exploring sectors. A ship (`Ship` class) has fuel, crew, and cargo capacity. Sectors (`Sector` class) have risk level, resources, and events. Using `ArrayLists`, players manage multiple ships and discovered sectors. Each turn they choose missions (explore, mine, trade) and resolve outcomes. Ships can be lost, upgraded, or expanded.

Required implementation & minimum features:

- Use **at least 3 classes**: `Ship`, `Sector`, `Fleet` (or `Game`), plus `Main`.
 - Use `ArrayList<Ship>` for player's fleet and `ArrayList<Sector>` for known sectors.
 - Each turn/day:
 - Allow player to assign ships to sectors for specific missions (explore, mine, trade).
 - Reduce fuel and calculate mission outcome (resources gained, damage, loss).
 - Implement **resources**: at least fuel and some kind of space-credits.
 - Implement **ship upgrade or new ship purchase**.
 - Provide a **status menu**:
 - View fleet, resources, and discovered sectors.
 - Define **win condition** (e.g., accumulate certain resources or find a “lost colony” sector) and **lose condition** (e.g., all ships lost or no fuel).
-

11. Text-Based Sports League Manager

Players manage a sports team. Teams (`Team` class) and players (`Player` class) are stored in `ArrayLists`. The system simulates a full season: fixtures, match results, standings. For each match, simple simulation logic uses team strength (based on players' stats) plus randomness. Players can transfer players between teams, train players (increasing stats), and adjust basic tactics.

Required implementation & minimum features:

- Use **at least 3 classes**: `Player`, `Team`, `League` (or `Game`), plus `Main`.
 - Use `ArrayList<Player>` in each team and `ArrayList<Team>` in the league.
 - Initialize a small league with several teams.
 - Implement **match simulation**:
 - Calculate each team's score based on team strength + randomness.
 - Generate a **season schedule** (each team plays others at least once).
 - Provide menu options to:
 - View team details and players.
 - Train players (increase stats).
 - Transfer players between teams (simple version is fine).
 - Track **standings** (points, wins, losses) and show a table.
 - End season and declare a **champion**.
-

12. Mystery Mansion Adventure

Players explore a mansion with multiple rooms and NPCs. Rooms (`Room` class) can store items (`Item` class) and characters (`NPC` class) in `ArrayLists`. The story: solve a mystery by visiting rooms, talking to NPCs, and collecting clues. The game uses commands like “GO kitchen,” “TAKE key,” “TALK butler.” Each action triggers methods that update state and story progress. Multiple endings are possible based on clues found.

Required implementation & minimum features:

- Use **at least 4 classes**: `Room`, `Item`, `NPC`, `Game` (or `World`), plus `Main`.
 - Use `ArrayList<Item>` in rooms and in the player’s inventory; optional `ArrayList<NPC>` per room.
 - Define a fixed set of rooms (at least 6–8) and connections between them.
 - Implement a **command system**:
 - Basic commands: GO (move), LOOK, TAKE, DROP, TALK, INVENTORY.
 - Implement **clue collection**:
 - Certain items or NPC dialogues count as clues.
 - Track **player progress**:
 - Based on collected clues and visited rooms.
 - Implement at least **one winning ending** (correctly accusing the culprit) and **one losing ending** (accusing wrong person or running out of turns).
-

13. Zombie Apocalypse Squad Tactics

Players control a small squad of survivors (`Survivor` class) trying to escape a city. Each survivor has skills (medic, fighter, mechanic). Locations (`Location` class) have danger level and resources (food, ammo, tools) stored in `ArrayLists`. Each day, players choose which location to visit and which survivors to send. Events simulate encounters and random risks.

Required implementation & minimum features:

- Use **at least 3 classes**: `Survivor`, `Location`, `Game` (or `City`), plus `Main`.
 - Use `ArrayList<Survivor>` for squad and `ArrayList<Location>` for known places.
 - Each day:
 - Show a list of locations and their basic info (danger, possible loot).
 - Player selects a destination and which survivors go.
 - Simulate **encounters**:
 - Use survivor skills and randomness to determine success/failure.
 - Update HP, ammo, food, and tools.
 - Track **resources** (food, ammo) and make them affect survival.
 - Show **daily status**: survivors' HP, resources, days passed.
 - Define **win condition** (reach extraction location or survive N days) and **lose condition** (all survivors dead or no food).
-

14. Rogue Merchant Caravan

Players run a traveling merchant caravan moving between towns. The caravan (`Caravan` class) has an `ArrayList` of items (`Item` class) and guards (`Guard` class). Each town has different prices and events; players decide what to buy/sell, which route to take, and whether to hire more guards. Random events like bandit attacks, weather, and market crashes affect progress.

Required implementation & minimum features:

- Use **at least 3 classes**: `Caravan`, `Item`, `Guard`, plus `Main` (and optional `Town`).
 - Use `ArrayList<Item>` for caravan inventory and `ArrayList<Guard>` for guards.
 - Define several **towns** with different buy/sell prices.
 - Each turn:
 - Choose next town to travel to.
 - Handle travel events (bandit attack, storm, nothing).
 - Implement **buy/sell system**:
 - Show item prices in each town.
 - Adjust caravan gold and inventory.
 - Implement **combat/resolution** for bandit attacks:
 - Use guards' strength and randomness.
 - Define **goal** (e.g., reach certain amount of gold after X trips) and **lose condition** (gold $\leq$ 0 or caravan destroyed).
-

15. Hacker vs. Firewall: Terminal Strategy Game

Players act as a hacker trying to penetrate multiple firewalls. Firewalls (`Firewall` class) have security levels, weaknesses, and modules stored in an `ArrayList`. The player builds an arsenal of tools (`Tool` class) like “BruteForce,” “Phishing,” “SQLInjection” with different power and cost. Each level is a sequence of firewalls where the player chooses which tools to use. Success is calculated via formulas + randomness. Resources (time, energy) are limited, requiring planning.

Required implementation & minimum features:

- Use **at least 3 classes**: `Firewall`, `Tool`, `Game` (or `HackSession`), plus `Main`.
 - Use `ArrayList<Firewall>` for firewalls in a level and `ArrayList<Tool>` for available tools.
 - Each firewall must have at least: security level, type, and weakness.
 - Each tool must have at least: name, power, cost, and specialty (which firewall types it's strong against).
 - Implement **turn-based hacking attempts**:
 - Player chooses which tool to apply to the current firewall.
 - Calculate success chance based on tool power, firewall security, weaknesses, and randomness.
 - Track **limited resources**: attempts, time, or energy.
 - Define **levels**:
 - At least a sequence of several firewalls per level.
 - Implement **win/lose condition**:
 - Win: break through all firewalls in a level before running out of resources.
 - Lose: resources reach zero.
-